

В Haskell списък е хомогенна структура, която се дефинира рекурсивно:

- `[]` празен списък е списък
- `(x:xs)` е списък когато `x` е глава (1 елемент) и `xs` е списък.

Понятие е хомогенен ако `x::a`, то `xs::[a]`. (Възможно е да имаме списъци от символи, например `"Hello" = ['H','e','l','l','o']`)

Основни операции върху списъци в Haskell са:

- `head` - връща първия елемент на списъка (главата)
- `tail` - връща опашката на списъка (нак е списък от дефиницията)
- `null` - проверява дали списъка е празен
- `++` - конкатенира два списъка
- `length` - връща дължината (брой на елементите) на списъка
- `reverse` - обръща дадения списък
- `take x lst` - взема първите `x` елемента на `lst`, а ако дължината на `lst` е по-малка от `x`, връща целия списък
- `drop x lst` - връща `lst` без първите `x` елемента, а ако `x` е по-голям от дължината на `lst`, връща празен списък
- `elem x lst` - проверява дали `x` е елемент на `lst`

Примери:

`head [1,2,3]` връща 1

`tail [1,2,3]` връща `[2,3]`

`null [1,2,3]` връща `False` `null []` връща `True`

`[1]++[2,3]` връща `[1,2,3]`

`length [1,2,3]` връща 3 `length []` връща 0

`reverse [1,2,3]` връща `[3,2,1]`

`take 2 [1,2,3]` връща `[1,2]` `take 20 [1,2,3]` връща `[1,2,3]`

`drop 2 [1,2,3]` връща `[3]` `drop 20 [1,2,3]` връща `[]`

`elem 2 [1,2,3]` връща `True` `elem 4 [1,2,3]` връща `False`

Функции от по-висок ред за работа със списъци

Функции от по-висок ред са функции които приемат като аргумент функция или брояч функция. Например можем да си дефинираме compose, която работи подобно на `.` в Haskell

$\text{compose } f\ g = \lambda x \rightarrow f(g\ x)$ - взема f и g - функции на един аргумент и брояч функция - композицията им

Основни функции за работа със списъци са

$\text{map } f\ \text{lst}$ - брояч списък от елементите на lst , но f е приложен върху тях.

$\text{map } (\lambda x \rightarrow x * x)\ [1,2,3]$ брояч $[1,4,9]$

$\text{filter } p\ \text{lst}$ - брояч списък от елементите на lst , за които предикатът p е верен.

$\text{filter odd } [1,2,3]$ брояч $[1,3]$

$\text{foldl } op\ init\ \text{lst}$ - брояч резултата от ляво свиване на lst с операция op и начална стойност $init$

$\text{foldl } (\lambda x\ y \rightarrow x + y)\ 0\ [1,2,3]$ брояч 6

$\text{foldr } op\ init\ \text{lst}$ - брояч резултатите от дясно свиване на lst с операция op и начална стойност $init$

$\text{foldr } (\lambda x\ y \rightarrow x + y)\ 0\ [1,2,3]$ брояч 6

Въпреки че в примерите foldl и foldr върнат едни и същи резултати, работят по различен начин, което има значение ако op не е асоциативна

• $\text{foldr } op\ nv\ (x:xs)$ прави $x\ 'op'\ \text{foldr } op\ nv\ xs$, т.е. прави:

$$x_1\ op\ (x_2\ op\ (x_3\ op\ (\dots (x_n\ op\ nv))) \dots)$$

• $\text{foldl } op\ nv\ (x:xs)$ прави $\text{foldl } op\ (x\ 'op'\ nv)\ xs$, т.е. прави:

$$(\dots (nv\ 'op'\ x_1)\ 'op'\ x_2) \dots 'op'\ x_n$$

Вспомогательные функции на списках и HashSet можно переписать так:

map:

$\text{map} _ [] = []$

$\text{map } f \ (x:xs) = f \ x : \text{map } f \ xs$

$\text{filter} _ [] = []$

$\text{filter } p \ (x:xs)$

$\mid p \ x = x : \text{filter } p \ xs$

$\text{otherwise} = \text{filter } p \ xs$

$\text{foldl} _ nv \ [] = nv$

$\text{foldl } op \ nv \ (x:xs) = \text{foldl } op \ (x \text{ `op' } nv) \ xs$

$\text{foldr} _ nv \ [] = nv$

$\text{foldr } op \ nv \ (x:xs) = x \text{ `op' } \text{foldr } op \ nv \ xs$

Основные операции с lazy list можно переписать так:

$\text{head } (h:_) = h$

$\text{take } 0 _ = []$

$\text{tail } (-:t) = t$

$\text{take} _ [] = []$

$\text{take } n \ (x:xs) = x : \text{take } (n-1) \ xs$

$\text{null} [] = \text{True}$

$\text{null} _ = \text{False}$

$\text{drop } 0 \ lst = lst$

$\text{drop} _ [] = []$

$\text{length} [] = 0$

$\text{drop } n \ (x:xs) = \text{drop } (n-1) \ xs$

$\text{length } (x:xs) = 1 + \text{length } xs$

$\text{elem} _ [] = []$

$\text{reverse} [] = []$

$\text{elem } x \ (h:t) = x == h \parallel \text{elem } x \ t$

$\text{reverse } (x:xs) = \text{reverse } xs ++ [x]$

При стриктното оценяване аргументите се оценяват преди извикването. Така работи Scheme (с някои изключения като например `define`, `if`, `cond`, `lambda`)

При ленивото оценяване аргументите се оценяват едва когато потребват.

Така работи Haskell, а от Теорема за нормализация знаем, че ако някоя стратегия на оценяване стигне до резултат, то и ленивото оценяване стигне до същия резултат.

Поневиде Haskell оценява лениво, $(x:xs)$ отлага и главата и опашката, те са обобщения за глава и опашка. Така всеки списък в Haskell е `pattern`, който няма проблем да бъде безкраен.

Haskell

`first x y = x`

`first 1 (1 `div` 0)` -- връща 1, а `1 `div` 0` не се оценява

Scheme

`(define (first x y) x)`

`(first 5 (1 1 0))` ; грешка, Scheme оценява стриктно и ще оцени `(1 0)` дори и да не му трябва (както е в случая)

`ones = 1:ones`

`take 5 ones` -- връща `[1,1,1,1,1]`

`length ones` -- зазукля, защото `ones` е безкраен

Генериране на безкрайни потоци

`nats = [0...]` -- генерира естествените числа

Може и така (без имитативна закъс)

`enumFrom from = from: enumFrom (from + 1)`

`nats = enumFrom 0`

`repeat x = x: repeat x` -- `[x,x,x,...]`

(функциите на безкрайни списъци трябва да се вичкава, защото извикват

като тези

map square nats

length nats

foldr (+) 0 nats

foldl (+) 0 nats

няма да завършат. Това обаче не означава че са безполезни, защото

можем да вземем крайна част от списъка, например

map (\x -> x+1) (take 10 nats) -- заберува и връща [1,2,3,4,5,6,7,8,9,10]

take и take

take 10 \$ map (\x -> x+1) nats -- връща същия резултат като горното

Но за fold-овете обаче трябва да се вичкава защото:

- foldl никога няма да завърши на безкраен списък

- foldr да работи само ако операцията не форсира десния аргумент, напр.

foldr (\x y -> x*x:y) [] nats

Функции които резултат на това да обхождат целия списък, за да дават

резултат няма да завършат

length nats -- брои ∞ забвизки

sum nats -- сумира ∞ забвизки

В Haskell има и още една много полезна нотация, която много прилича на дефиниция на множество

odds = [x | x < nats, x `mod` 2 == 1] -- odds = {x | x ∈ $\mathbb{N}$, x ≡ 1 (mod 2)}

Така можем много лесно да направим безкраен списък с всички прости числа

sieve (x:xs) = x : sieve [y | y < xs, y `mod` x /= 0]

primes = sieve [2..]